



Mundial

MANAGER

Crowd Management System for FIFA World Cup 2026

Feature Implementation Report

Flutter + Firebase + Provider Architecture

Platform: Android & iOS

Version: 1.0.0

Table of Contents

1 System Architecture

2 Authentication System

2.1 Login with Email & Password

2.2 User Registration with Role Selection

2.3 Password Reset

3 Fan Dashboard

3.1 View Venue Map

3.2 Receive Live Alerts

3.3 Share Location

3.4 Report Incident

4 Security Team Dashboard

4.1 Receive & Respond to Alerts

4.2 Report Incidents

4.3 Monitor Crowd Density

5 Emergency Services Dashboard

5.1 View Incident Locations on Map

5.2 Update Response Status

5.3 Communicate with Organizers

6 Event Organizer Dashboard

6.1 View Crowd Movement

6.2 Send Safety Alerts

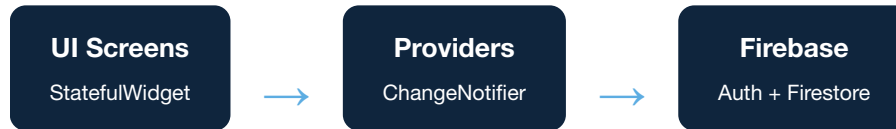
6.3 View Analytics & Reports

6.4 Manage Roles & Zones

7 Demo Accounts

1. System Architecture

Mundial uses a **3-layer architecture** with the Provider pattern for state management:



State Management Providers

PROVIDER	RESPONSIBILITY	FIRESTORE COLLECTIONS
<code>AuthProvider</code>	Login, Register, Logout, Location Sharing	users
<code>CrowdProvider</code>	Zone data, Crowd density, Real-time heatmap	zones, crowd_density
<code>IncidentProvider</code>	Report & view incidents, Update status	incidents
<code>AlertProvider</code>	Send, Receive, Resolve safety alerts	alerts
<code>StaffProvider</code>	Staff assignment to zones	staff_assignments, users
<code>AnalyticsProvider</code>	Charts, Reports, Statistics	events, incidents, crowd_density
<code>MessageProvider</code>	Real-time messaging channels	messages

Technology Stack

LAYER	TECHNOLOGY
Frontend	Flutter (Dart) — cross-platform Android & iOS
State Management	Provider with ChangeNotifier pattern
Authentication	Firebase Authentication (email/password)
Database	Cloud Firestore (NoSQL, real-time streams)
Storage	Firebase Storage (incident images)

Push Notifications	Firebase Cloud Messaging (FCM)
Maps	flutter_map + OpenStreetMap tiles + LatLng2
Charts	fl_chart library
Local Storage	SharedPreferences (settings, session)

2. Authentication System

The auth system supports 4 user roles with email verification, login logout after 5 failed attempts, session timeout, and Remember Me functionality.

2.1 Login with Email & Password

Login Screen

lib/screens/auth/login_screen.dart → lib/providers/auth_provider.dart → lib/services/auth_service.dart

 Screenshot: Login Screen with email field, password field, Remember Me checkbox, Forgot Password link, Login button, and social login buttons

```
// login_screen.dart - Handles login with validation & routing
Future<void> _handleLogin() async {
  if (!_formKey.currentState!.validate()) return;

  final authProvider = Provider.of<AuthProvider>(context, listen: false);

  final success = await authProvider.login(
    _emailController.text.trim(),
    _passwordController.text,
  );

  if (success) {
    // Save remembered email if checkbox is checked
    if (_rememberMe) {
      await prefs.setBool(AppConstants.keyRememberMe, true);
      await prefs.setString(AppConstants.keyRememberedEmail,
        _emailController.text.trim());
    }

    // Route based on email verification status
    if (authProvider.needsEmailVerification) {
      context.go('/email-verification');
    } else {
      context.go(AppRouter.getDashboardRoute(authProvider.userRole!));
    }
  }
}
```

```
// auth_provider.dart - Login with lockout protection
Future<bool> login(String email, String password) async {
  // Check if account is locked (5 failed attempts = 15 min lockout)
  if (isLockedOut) {
    _errorMessage = 'Account locked. Try again in $remaining minutes.';
    return false;
  }

  final user = await _authService.signInWithEmailAndPassword(
    email: email, password: password,
  );

  if (user != null) {
    _loginAttempts = 0; // Reset on success
    _currentUser = user;
    await _saveUserToPrefs();
    await _initializeNotifications();
    return true;
  }
}
```

```
// auth_service.dart - Firebase Auth sign-in + Firestore user data
Future<User?> signInWithEmailAndPassword({
  required String email,
  required String password,
}) async {
  final credential = await _auth.signInWithEmailAndPassword(
    email: email, password: password,
  );

  // Update last login timestamp in Firestore
  await _firestore.collection('users').doc(credential.user!.uid)
    .update({'lastLogin': FieldValue.serverTimestamp()});

  // Fetch user profile from Firestore
  final userDoc = await
    _firestore.collection('users').doc(credential.user!.uid).get();
  return User.fromJson({'id': credential.user!.uid, ...userDoc.data()!});
}
```

2.2 User Registration with Role Selection

Register Screen

lib/screens/auth/register_screen.dart

 Screenshot: Register Screen with Name, Email, Password, Confirm Password fields, and Role dropdown (Fan, Organizer, Security, Emergency)

```
// register_screen.dart - Registration with role selection
Future<void> _handleRegister() async {
  final success = await authProvider.register(
    email: _emailController.text.trim(),
    name: _nameController.text.trim(),
    password: _passwordController.text,
    role: _selectedRole!, // fan, organizer, security, emergency
  );

  if (success) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Registration successful!')),
    );
    context.pop(); // Return to login
  }
}
```

```
// auth_service.dart - Creates Firebase Auth + Firestore user document
Future<User?> registerWithEmailAndPassword({...}) async {
  final credential = await _auth.createUserWithEmailAndPassword(
    email: email, password: password,
  );

  // Send email verification
  await credential.user!.sendEmailVerification();

  // Create user profile in Firestore
  await _firestore.collection('users').doc(credential.user!.uid).set({
    'email': email, 'name': name, 'role': role,
    'createdAt': FieldValue.serverTimestamp(),
    'isActive': true, 'emailVerified': false,
  });
}
```

2.3 Password Reset

Forgot Password Dialog

lib/screens/auth/login_screen.dart (line 90)

```
// Sends password reset email via Firebase Auth
final success = await authProvider.resetPassword(email);
// → auth_service.dart calls: _auth.sendPasswordResetEmail(email: email);
```

3. Fan Dashboard

FAN

Login: `fan@test.com` / `password123`

3.1 View Venue Map

Interactive venue map with real-time crowd heatmap, zone details, and Points of Interest

`lib/screens/fan/venue_map_screen.dart`



Screenshot: Map tab showing color-coded zone heatmap (green=safe, yellow=moderate, red=critical) with POI markers for First Aid, Food, Prayer Room, Exit, Entrance

```
// venue_map_screen.dart - Real-time heatmap from Firestore
CrowdHeatmap(
  crowdData: crowdProvider.crowdData,    // Live from Firestore
  zones: crowdProvider.allZones,         // 7 zones from Firestore
  onZoneTap: (zone) {
    _showZoneDetails(context, zone.id, crowdProvider);
  },
),
```

```
// Zone tap shows live occupancy data
void _showZoneDetails(context, zoneId, crowdProvider) {
  final zone = crowdProvider.getZone(zoneId);
  final density = crowdProvider.getZoneDensity(zoneId);

  showModalBottomSheet(
    builder: (_) => Column(children: [
      Text(zone.name),
      Text('Population: ${density.currentPopulation}/${density.capacity}'),
      Text('Occupancy: ${density.occupancyPercentageRounded}%'),
      Text('Density: ${density.densityPerSqMeter.toStringAsFixed(1)} p/m²'),
    ]),
  );
}
```


3.2 Receive Live Alerts

Real-time alert notifications filtered by fan role with Acknowledge and Notify Emergency actions

lib/screens/fan/notifications_screen.dart

 Screenshot: Alerts tab showing alert cards with severity indicators (Critical=red, Warning=orange, Info=blue) and time ago labels

```
// notifications_screen.dart - Loads alerts filtered for fan role
Consumer<AlertProvider>(  
  builder: (context, alertProvider, _) {  
    final allAlerts = alertProvider.getAlertsForRole('fan');  
  
    return ListView.builder(  
      itemCount: allAlerts.length,  
      itemBuilder: (context, index) => _AlertCard(  
        alert: allAlerts[index],  
        onTap: () => _showAlertDetails(context, allAlerts[index], alertProvider),  
      ),  
    );  
  },  
)
```

```
// Alert detail sheet - Fan can Acknowledge, Notify Emergency, or Mark Resolved
ElevatedButton(  
  onPressed: () async {  
    await alertProvider.dismissAlert(alert.id); // Acknowledge  
  },  
  child: Text('Acknowledge'),  
)  
ElevatedButton(  
  onPressed: () async {  
    await alertProvider.sendEmergencyAlert( // Notify Emergency  
      eventId: alert.eventId,  
      createdBy: user.id,  
      message: 'Fan reported emergency: ${alert.message}',  
    );  
  },  
  child: Text('Notify Emergency'),  
)
```

3.3 Share Location

Toggle location sharing for safety — updates Firestore & uses device GPS

lib/screens/fan/fan_dashboard.dart (line 572)

 Screenshot: Home tab showing "Share My Location" button (green border when ON, gray when OFF)

```
// fan_dashboard.dart - Location sharing toggle
Consumer<AuthProvider>(  
  builder: (context, authProvider, _) {  
    final isSharing = authProvider.currentUser?.locationSharingEnabled ?? false;  
  
    return GestureDetector(  
      onTap: () => authProvider.toggleLocationSharing(),  
      child: Container(  
        color: isSharing ? AppColors.green : Colors.white10,  
        child: Text(isSharing ? 'Location Sharing: ON' : 'Share My Location'),  
      ),  
    );  
  },  
)
```

```
// auth_provider.dart - Backend toggle
Future<void> toggleLocationSharing() async {  
  final newValue = !_currentUser!.locationSharingEnabled;  
  if (newValue) {  
    final success = await _locationService.startSharing(_currentUser!.id);  
    if (!success) { _errorMessage = 'Location permission denied.'; return; }  
  } else {  
    await _locationService.stopSharing();  
  }  
  await updateProfile(locationSharingEnabled: newValue);  
}
```

3.4 Report Incident

Full incident report form with type selection, severity level, description, and photo upload to Firebase Storage

lib/screens/common/report_incident_screen.dart

 Screenshot: Report Incident screen showing incident type chips (Medical, Security, Overcrowding, Facility), severity selector (Low, Medium, High, Critical), description field, and photo upload area

```

// report_incident_screen.dart - Submit incident to Firestore
Future<void> _submitReport() async {
  // Upload images to Firebase Storage
  if (_images.isNotEmpty) {
    imageUrls = await StorageService().uploadIncidentImages(
      incidentId: incidentId, imageFiles: imageFiles,
    );
  }

  // Submit to Firestore via IncidentProvider
  final success = await incidentProvider.reportIncident(
    eventId: event?.id ?? '',
    reportedBy: user.id,
    reportedByName: user.name,
    location: incidentLocation,
    type: _selectedType, // medical, security, overcrowding
    description: _descriptionController.text.trim(),
    severity: _selectedSeverity, // low, medium, high, critical
    imageUrls: imageUrls,
  );
}

```

4. Security Team Dashboard

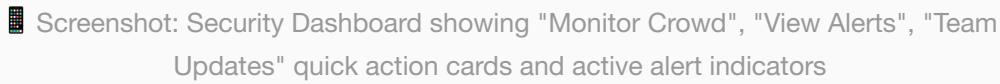
SECURITY

Login: `security@test.com` / `password123`

4.1 Receive & Respond to Alerts

Dashboard with quick action cards, active alerts list, and alert detail sheet with Mark Resolved action

`lib/screens/security/security_dashboard.dart`

 Screenshot: Security Dashboard showing "Monitor Crowd", "View Alerts", "Team Updates" quick action cards and active alert indicators

```
// security_dashboard.dart - Quick action cards navigate to real screens
_QuickActionCard(
  title: 'Monitor Crowd',  onTap: () => onNavigate(1),  // Zones tab
),
_QuickActionCard(
  title: 'View Alerts',    onTap: () => onNavigate(2),  // Alerts tab
),
_QuickActionCard(
  title: 'Team Updates',  onTap: () => context.push('/communication'),
),
```

```
// Alert detail sheet with resolve action
ElevatedButton(
  onPressed: () async {
    final success = await alertProvider.resolveAlert(alert.id);
    Navigator.pop(context);
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(success ? 'Alert resolved' : 'Failed')),
    );
  },
  child: Text('Mark Resolved'),
),
```

4.2 Report Incidents

Report button opens shared ReportIncidentScreen (same form as Fan, line 3.4)

`lib/screens/common/report_incident_screen.dart`

```
// security_dashboard.dart - Report button in monitoring panel
CustomButton.warning(
  text: 'Report',
  icon: Icons.warning_amber,
  onPressed: () => Navigator.push(context,
    MaterialPageRoute(builder: (_) => const ReportIncidentScreen()),
  ),
),
```

4.3 Monitor Crowd Density

Zones tab shows real-time density per zone; Map tab shows heatmap with incident markers

lib/screens/security/security_dashboard.dart

 Screenshot: Zones tab showing zone cards with occupancy bars (e.g., North Stand 85%, Food Court 95%) and color-coded status

```
// security_dashboard.dart - _ZonesTab with real Firestore data
Consumer<CrowdProvider>(
  builder: (context, crowdProvider, _) {
    final zones = crowdProvider.allZones;

    return ListView.builder(
      itemCount: zones.length,
      itemBuilder: (context, index) {
        final zone = zones[index];
        final density = crowdProvider.getZoneDensity(zone.id);
        final occupancy = density?.occupancyPercentageRounded ?? 0;

        return GlassCard(child: Column(children: [
          Text(zone.name),
          Text('${density?.currentPopulation ?? 0} / ${zone.capacity}'),
          LinearProgressIndicator(value: occupancy / 100),
          Text('$occupancy% capacity'),
        ]));
      },
    );
  },
),
```

5. Emergency Services Dashboard

EMERGENCY

Login: `emergency@test.com` / `password123`

5.1 View Incident Locations on Map

FlutterMap with incident markers colored by severity, tappable for navigation to details

`lib/screens/emergency/emergency_dashboard.dart`

 Screenshot: Emergency Map tab showing venue map with colored incident markers (red=critical, orange=high, yellow=medium) at real GPS coordinates

```
// emergency_dashboard.dart - _MapTab with incident markers
FlutterMap(
  options: MapOptions(initialCenter: venueCenter, initialZoom: 16.0),
  children: [
    TileLayer(urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png'),
    MarkerLayer(
      markers: incidentProvider.activeIncidents.map((incident) {
        return Marker(
          point: LatLng(incident.latitude, incident.longitude),
          child: GestureDetector(
            onTap: () {
              // Navigate to Alerts tab for incident details
              dashState?.setState(() => dashState._selectedIndex = 2);
            },
            child: IncidentMarkerWidget(
              severity: incident.severity, type: incident.type,
            ),
          ),
        );
      }).toList(),
    ),
  ],
),
```

5.2 Update Response Status

Dashboard metrics show real dispatched count; evacuation dialog sends live alert to all users

```
lib/screens/emergency/emergency_dashboard.dart
```

 Screenshot: Emergency Dashboard tab showing Active Incidents count, Teams Dispatched count, and INITIATE EVACUATION button

```
// emergency_dashboard.dart - Real dispatched teams count
Text(
  '${incidentProvider.getIncidentsByStatus("dispatched").length}',
  style: GoogleFonts.montserrat(fontSize: 28, fontWeight: FontWeight.bold),
),
Text('Teams Dispatched'),
```

```
// Evacuation sends real emergency alert to Firestore
ElevatedButton(
  onPressed: () async {
    final success = await alertProvider.sendEmergencyAlert(
      eventId: eventId,
      createdBy: user.id,
      createdByName: user.name,
      message: 'EVACUATION ORDER: All attendees must evacuate immediately.',
    );
  },
  child: Text('INITIATE EVACUATION'),
),
```

5.3 Communicate with Organizers

Real-time messaging via Communication Hub with role-based channels

```
lib/screens/common/communication_hub_screen.dart
```

 Screenshot: Communication Hub showing channels list (security-ops, emergency-ops, all-staff) and message thread with sender name, role badge, and timestamps

```
// communication_hub_screen.dart - Send message to Firestore
Future<void> _sendMessage() async {
  final success = await messageProvider.sendMessage(
    content: text,
    senderId: user.id,
    senderName: user.name,
    senderRole: user.role,      // Shows role badge in chat
    type: _selectedMessageType, // text, alert, incident_update
  );
}
```


6. Event Organizer Dashboard

ORGANIZER

Login: `organizer@test.com` / `password123`

6.1 View Crowd Movement

Crowd monitoring tab with heatmap and zone density cards showing live occupancy

`lib/screens/organizer/organizer_dashboard.dart` (Crowd tab)

 Screenshot: Organizer Crowd tab with heatmap at top and zone density list below
(North Stand 85%, VIP 92%, Food Court 95%)

```
// organizer_dashboard.dart - _CrowdTab with real-time data
Consumer<CrowdProvider> (
  builder: (context, crowdProvider, _) {
    return Column(children: [
      // Real-time heatmap
      CrowdHeatmap(
        crowdData: crowdProvider.crowdData,
        zones: crowdProvider.allZones,
      ),
      // Zone density cards from Firestore
      ...crowdProvider.allZones.map((zone) {
        final density = crowdProvider.getZoneDensity(zone.id);
        return _ZoneDensityCard(
          zoneName: zone.name,
          currentCount: density?.currentPopulation ?? 0,
          capacity: zone.capacity,
        );
      }),
    ]);
  },
)
```

6.2 Send Safety Alerts

Create alert dialog with type, message, severity, and target role selection — saved to Firestore

`lib/screens/organizer/organizer_dashboard.dart` (Alerts tab)

 Screenshot: Send Alert dialog showing alert type dropdown, message text area, severity selection, target role checkboxes (Fans, Security, Emergency), and Send button

```
// organizer_dashboard.dart - Send alert to Firestore
final success = await alertProvider.sendAlert(
  eventId: _currentEventId!,
  createdBy: user.id,
  createdByName: user.name,
  type: selectedType,          // congestion, safety, emergency, info
  message: messageController.text,
  severity: selectedSeverity, // info, warning, critical
  targetRoles: targetRoles,    // ['fan', 'security', 'emergency']
);
```

6.3 View Analytics & Reports

Analytics screen with charts (fl_chart), date range picker, incident statistics, and crowd trend data

lib/screens/organizer/analytics_screen.dart

 Screenshot: Analytics screen showing line chart for crowd trends, bar chart for incidents by type, and summary statistics cards

```
// analytics_screen.dart - Charts with data from AnalyticsProvider
Future<void> _initializeData() async {
  final event = await DatabaseService().getCurrentActiveEvent();
  await analyticsProvider.initialize(event?.id ?? '');
}

// Date range picker for filtering data
final picked = await showDateRangePicker(
  context: context,
  firstDate: DateTime.now().subtract(const Duration(days: 365)),
  lastDate: DateTime.now(),
);
```

6.4 Manage Roles & Zones

Staff Management with 3 tabs: All Staff (with assign dialog), Active Assignments, Zone Coverage

```
lib/screens/organizer/staff_management_screen.dart
```

 Screenshot: Staff Management — All Staff tab showing security & emergency personnel with Assign/Reassign buttons; Zones tab showing zone names with assigned staff count

```
// staff_management_screen.dart - Assign staff to zone
void _showAssignDialog(context, staff) {
  final zones = Provider.of<CrowdProvider>(context).allZones;

  // Zone dropdown from Firestore
  DropdownButton<String>(
    hint: Text('Choose a zone'),
    items: zones.map((zone) {
      final count = staffProvider.getAssignmentsForZone(zone.id).length;
      return DropdownMenuItem(
        value: zone.id,
        child: Text('${zone.name} - $count staff'),
      );
    }).toList(),
  ),

  // Assign button saves to Firestore
  CustomButton.primary(
    text: 'Assign to Zone',
    onPressed: () async {
      await staffProvider.assignStaffToZone(
        staffId: staff.id, staffName: staff.name,
        zoneId: zone.id, zoneName: zone.name,
        assignedBy: authProvider.currentUser!.id,
      );
    },
  ),
}
```

7. Demo Accounts

The following pre-seeded accounts are available for testing all features:

ROLE	EMAIL	PASSWORD	FEATURES AVAILABLE
FAN	fan@test.com	password123	View venue map, Receive alerts, Share location, Report incidents
ORGANIZER	organizer@test.com	password123	Crowd monitoring, Send alerts, Analytics, Staff management, Create events
SECURITY	security@test.com	password123	Receive/respond alerts, Report incidents, Monitor crowd density, Communication hub
EMERGENCY	emergency@test.com	password123	View incident locations, Update response status, Evacuation alerts, Communication hub

Seed Data Summary

DATA TYPE	COUNT	DETAILS
Users	11	2 fans, 1 organizer, 5 security, 3 emergency
Venue	1	King Fahd International Stadium, Riyadh
Zones	7	North/South Stand, East/West Wing, VIP, Food Court, Main Entrance
Facilities	17	Restrooms, food vendors, first aid, exits, info desks, prayer rooms
Event	1	Saudi Arabia vs Mexico — FIFA World Cup 2026
Crowd Density	7	Varying levels: 45%–95% (safe to critical)
Incidents	8	All statuses: reported, dispatched, on_site, resolved

Alerts	7	All types: emergency, congestion, safety, info
Staff Assignments	7	4 security + 2 emergency assigned; 2 unassigned (available)
Messages	15	Across 3 channels: security-ops, emergency-ops, all-staff

Security Features

FEATURE	IMPLEMENTATION
Password Hashing	Firebase Auth (bcrypt internally)
Login Lockout	5 failed attempts → 15-minute lockout
Session Timeout	Auto-logout after 7 days inactivity
Email Verification	Sent on registration, checked on login
Remember Me	Email stored in SharedPreferences
Offline Support	User data cached in SharedPreferences
Input Validation	Email regex, password strength, name length
Firestore Rules	Role-based access control (RBAC)
Account Deletion	Firebase Auth + Firestore cleanup